

3Date: 21/04/2026

Tags:

Lista de Significados

- **RTOS** -> Real-Time Operating System
- **Framework** -> Software environment que é reutilizável, funciona como um esqueleto para aplicações. Possui bibliotecas, APIs e ferramentas para desenvolvimento.
- **Tasks** -> Funções independentes, cada task com a sua memória alocada e prioridade. Estados das tasks: Running, Ready, Block ou Suspended.
- **Scheduler** -> Decide qual das tasks correr baseando-se na prioridade de cada uma. Consegue interromper uma task de baixa prioridade para correr uma de alta prioridade.
- **Priorities** -> Números mais altos significa maior prioridade.

FreeRTOS

FreeRTOS providencia um framework para correr várias "tasks" em simultâneo, cada task possui a sua prioridade e horário de execução. Possui ferramentas como "queues" e "semaphores".

Tasks

Criar uma Task

- 1º : Declarar uma variável do tipo/class `TaskHandle_t` que serve como um ponteiro para a task. Usamos a variável para controlar a task (e.g resumir, parar ou apagar a task).
- 2º : Criar a task. Uma task, como exposto na lista de significados, nada mais é do que uma função, porém têm de obedecer a duas condições : Tem de dar return void; E tem de aceitar um único argumento que poderá ser usado para passar dados para a função.
- 3º : Para finalizar e usufruir da task, temos de designar uma core específico para correr a task. Para isto usamos uma função específica -> `xTaskCreatePinnedToCore()` . Esta função especifica as características da task, a sua função, nome, tamanho da stack, parâmetros, prioridade e o "task handle".

Criação de Task - Exemplo

```
1
2  /*
3   Rui Santos & Sara Santos – Random Nerd Tutorials
4   Complete project details at https://RandomNerdTutorials.com/esp32-freertos-
   arduino-tasks/
5   Permission is hereby granted, free of charge, to any person obtaining a copy of
   this software and associated documentation files.
6   The above copyright notice and this permission notice shall be included in all
   copies or substantial portions of the Software.
7  */
8  #define LED_PIN 2
9
```

```

10 // Declare task handle
11 TaskHandle_t BlinkTaskHandle = NULL;
12
13 void BlinkTask(void *parameter) {
14     for (;;) { // Infinite loop
15         digitalWrite(LED_PIN, HIGH);
16         Serial.println("BlinkTask: LED ON");
17         vTaskDelay(1000 / portTICK_PERIOD_MS); // 1000ms
18         digitalWrite(LED_PIN, LOW);
19         Serial.println("BlinkTask: LED OFF");
20         vTaskDelay(1000 / portTICK_PERIOD_MS);
21         Serial.print("BlinkTask running on core ");
22         Serial.println(xPortGetCoreID());
23     }
24 }
25
26 void setup() {
27     Serial.begin(115200);
28
29     pinMode(LED_PIN, OUTPUT);
30
31     xTaskCreatePinnedToCore(
32         BlinkTask,          // Task function
33         "BlinkTask",        // Task name
34         10000,              // Stack size (bytes)
35         NULL,               // Parameters
36         1,                  // Priority
37         &BlinkTaskHandle,   // Task handle
38         1                   // Core 1
39     );
40 }
41
42 void loop() {
43     // Empty because FreeRTOS scheduler runs the task
44 }

```

Resumir e Suspende uma Task

Para o propósito de resumir ou suspender uma task, o FreeRTOS providencia-nos com duas functions :

- `vTaskSuspend()` : Passamos como argumento a task handle;
- `vTaskResume()` : Passamos como argumento a task handle.

Criar e Correr múltiplas Task

A criação para uma ou mais tasks é igual, tirando o número de vezes que tem-se de repetir o processo.

Também podemos criar e correr diversas tasks usando os dois núcleos. Para este fim, precisamos de mudar uma das características especificadas quando chamamos a função

`xTaskCreatePinnedToCore()` . No último campo, específico do core, basta mudar para o número 0.

```

1  void setup() {
2      Serial.begin(115200);
3
4      xTaskCreatePinnedToCore(
5          Task1,                // Task function
6          "Task1",              // Task name
7          10000,                // Stack size (bytes)
8          NULL,                 // Parameters
9          1,                    // Priority
10         &Task1Handle,         // Task handle
11         1                      // Core 1
12     );
13
14     xTaskCreatePinnedToCore(
15         Task2,                // Task function
16         "Task2",              // Task name
17         10000,                // Stack size (bytes)
18         NULL,                 // Parameters
19         1,                    // Priority
20         &Task2Handle,         // Task handle
21         0                      // Core 0
22     );
23 }

```

Memória Usada

As tasks do FreeRTOS, na ESP32, usam dois tipos diferentes de memória (em simultâneo): heap e stack; Cada uma destas memórias possui propósitos diferentes.

- **Uso da Stack** -> Uma área de memória dedicada para cada task, pode guardar variáveis locais, informação sobre a function call e o estado da task em execução. A stack de cada task é criada ao mesmo tempo que a task, o que por sua vez implica que temos de alocar previamente o espaço necessário para executar a task. Existe uma função `uxTaskGetStackHighWaterMark()` que determina o espaço de memória não usado, daí conseguimos alocar uma stack mais exata ao fazer algumas iterações.
- **Uso da Heap** -> A heap é uma "memory pool" conjunta, dentro da SRAM da ESP32. Usada para memória dinâmica (e.g, task stack, buffers). À semelhança da stack a heap também possui uma função com o propósito de determinar o espaço não usado, `xPortGetFreeHeapSize()`

Semáforos

Semáforos são ferramentas de sinalização usadas para coordenar as tasks, são normalmente usados para indicar que um evento ocorreu ou que algo está disponível. Semáforos não podem transportar dados, só um sinal para sinalizar que algo aconteceu.

TASK 1**SEMAPHORE****TASK 2**

 **BLOCKED**
(waiting for semaphore)

1

0

2

Give to semaphore

1

Take from semaphore
 **TASK RUNS**

3

0

 **BLOCKED**
(waiting for semaphore)

Semaphore resets

Existem dois tipos de semáforos:

- **Semáforo Binário** -> Sinaliza um único evento. Serve como ferramenta de sincronização e pode ter valores como 0 (vazio) e 1 (cheio).
- **Semáforo de Contagem** -> Em vez de só monitorizar um evento, monitoriza N eventos (N sendo o número de eventos que o utilizador define). E funciona como uma sincronização digital.

Functions de um semáforo

- `xSemaphoreCreateBinary()` -> Cria uma semáforo binário e devolve um handle da classe `SemaphoreHandle_t` quando sucede na criação do semáforo, ou devolve NULL se não conseguir criar.
- `xSemaphoreCreateCounting()` -> Cria uma semáforo de contagem e devolve um handle da classe `SemaphoreHandle_t` quando sucede na criação do semáforo, ou devolve NULL se não conseguir criar. Para além disto, podemos passar o número máximo de eventos que ele conta (N) como argumento.
- `xSemaphoreTake(semaphore, timeout)` -> Usamos esta função dentro de uma task para esperar ou obter um semáforo. Num semáforo binário se o estado for 1, a tarefa obtém o semáforo e troca o estado para 0, e se o estado for 0, a tarefa espera pelo tempo definido (timeout) pelo semáforo. Por outro lado, num semáforo de contagem, quando o N é maior que 0 ele vai decrementando até chegar a 0, que manda a task esperar.
- `xSemaphoreGive()` -> Usada dentro de uma task para dar o semáforo a uma outra, fazendo com que uma possível task que estivesse à espera, obter o semáforo.
- `xSemaphoreGiveISR()` -> Mesma coisa que `xSemaphoreGive()` mas quando é usada numa interrupt.

Códigos de Exemplo

```
1  /*
2   Rui Santos & Sara Santos - Random Nerd Tutorials
3   Complete project details at https://RandomNerdTutorials.com/esp32-freertos-semaphores-arduino/
4   Permission is hereby granted, free of charge, to any person obtaining a copy of
   this software and associated documentation files.
5   The above copyright notice and this permission notice shall be included in all
   copies or substantial portions of the Software.
6  */
7  #define BUTTON_PIN 23
8  #define LED1_PIN 2    // Toggled LED
9  #define LED2_PIN 4    // Blinking LED
10
11 #define DEBOUNCE_DELAY 200
12
13 SemaphoreHandle_t buttonSemaphore = NULL;
14
15 volatile uint32_t lastInterruptTime = 0;
16
17 void IRAM_ATTR buttonISR() {
18     uint32_t currentTime = millis();
19     if (currentTime - lastInterruptTime > DEBOUNCE_DELAY) {
20         BaseType_t higherPriorityTaskWoken = pdFALSE;
21         xSemaphoreGiveFromISR(buttonSemaphore, &higherPriorityTaskWoken);
22         lastInterruptTime = currentTime;
23         if (higherPriorityTaskWoken) {
24             portYIELD_FROM_ISR();
25         }
26     }
27 }
28
29 void LEDToggleTask(void *parameter) {
30     pinMode(LED1_PIN, OUTPUT);
31     bool ledState = false;
32     for (;;) {
33         if (xSemaphoreTake(buttonSemaphore, portMAX_DELAY)) {
34             ledState = !ledState;
35             digitalWrite(LED1_PIN, ledState ? HIGH : LOW);
36             Serial.print("LEDToggleTask: LED1 ");
37             Serial.println(ledState ? "ON" : "OFF");
38         }
39     }
40 }
41
42 void LEDBlinkTask(void *parameter) {
43     pinMode(LED2_PIN, OUTPUT);
44     for (;;) {
45         digitalWrite(LED2_PIN, HIGH);
46         Serial.println("LEDBlinkTask: LED2 ON");
47         vTaskDelay(250 / portTICK_PERIOD_MS);
```

```

48     digitalWrite(LED2_PIN, LOW);
49     Serial.println("LEDBlinkTask: LED2 OFF");
50     vTaskDelay(250 / portTICK_PERIOD_MS);
51 }
52 }
53
54 void setup() {
55     Serial.begin(115200);
56
57     // Defining the button as an interrupt
58     pinMode(BUTTON_PIN, INPUT_PULLUP);
59     attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), buttonISR, FALLING);
60
61     buttonSemaphore = xSemaphoreCreateBinary();
62     if (buttonSemaphore == NULL) {
63         Serial.println("Failed to create semaphore!");
64         while (1);
65     }
66
67     xTaskCreatePinnedToCore(
68         LEDToggleTask,           // Task function
69         "LEDToggleTask",        // Task name
70         3000,                   // Stack size
71         NULL,                   // Task parameters
72         2,                      // Higher priority
73         NULL,                   // Task handle
74         1                       // Core ID
75     );
76
77     xTaskCreatePinnedToCore(
78         LEDBlinkTask,           // Task function
79         "LEDBlinkTask",        // Task name
80         3000,                   // Stack size
81         NULL,                   // Task parameters
82         1,                      // Medium priority
83         NULL,                   // Task handle
84         1                       // Core ID
85     );
86 }
87
88 void loop() {
89
90 }
91

```

Exemplo Semáforo de Contagem

```

1  /*
2   Rui Santos & Sara Santos - Random Nerd Tutorials
3   Complete project details at https://RandomNerdTutorials.com/esp32-freertos-semaphores-arduino/

```

4 Permission is hereby granted, free of charge, to any person obtaining a copy of
this software and associated documentation files.

5 The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

6 */

7 #define BUTTON_PIN 23

8 #define LED1_PIN 2 // Blinking LED

9 #define LED2_PIN 4 // Fading LED

11 #define DEBOUNCE_DELAY 200 // debounce for the pushbutton in milliseconds

13 #define SEMAPHORE_MAX_COUNT 5

15 SemaphoreHandle_t buttonSemaphore = NULL;

16 volatile uint32_t lastInterruptTime = 0;

18 void IRAM_ATTR buttonISR() {

19 uint32_t currentTime = millis();

20 if (currentTime - lastInterruptTime > DEBOUNCE_DELAY) {

21 BaseType_t higherPriorityTaskWoken = pdFALSE;

22 if (xSemaphoreGiveFromISR(buttonSemaphore, &higherPriorityTaskWoken) == pdTRUE)

{

23 Serial.println("buttonISR: Gave semaphore token");

24 } else {

25 Serial.println("buttonISR: Semaphore full");

26 }

27 lastInterruptTime = currentTime;

28 if (higherPriorityTaskWoken) {

29 portYIELD_FROM_ISR();

30 }

31 }

32 }

34 void LEDBlinkTask(void *parameter) {

35 pinMode(LED1_PIN, OUTPUT);

36 for (;;) {

38 // Get and print the current semaphore count

39 UBaseType_t count = uxSemaphoreGetCount(buttonSemaphore);

40 Serial.print("LEDBlinkTask: Current semaphore count = ");

41 Serial.println(count);

43 if (xSemaphoreTake(buttonSemaphore, portMAX_DELAY)) {

44 Serial.println("LEDBlinkTask: Blinking LED1 for button press");

45 vTaskDelay(500 / portTICK_PERIOD_MS);

46 digitalWrite(LED1_PIN, HIGH);

47 vTaskDelay(1000 / portTICK_PERIOD_MS);

48 digitalWrite(LED1_PIN, LOW);

49 vTaskDelay(500 / portTICK_PERIOD_MS);

50 }

51 }

52 }

```

53
54 void LEDFadeTask(void *parameter) {
55     pinMode(LED2_PIN, OUTPUT);
56     for (;;) {
57         // Fade up (0 to 255)
58         for (int duty = 0; duty <= 255; duty += 5) {
59             analogWrite(LED2_PIN, duty);
60             if (duty % 50 == 0) { // Print every 10th step
61                 Serial.print("LEDFadeTask: Fading LED2, duty=");
62                 Serial.println(duty);
63             }
64             vTaskDelay(50 / portTICK_PERIOD_MS);
65         }
66         // Fade down (255 to 0)
67         for (int duty = 255; duty >= 0; duty -= 5) {
68             analogWrite(LED2_PIN, duty);
69             if (duty % 50 == 0 || duty == 255) {
70                 Serial.print("LEDFadeTask: Fading LED2, duty=");
71                 Serial.println(duty);
72             }
73             vTaskDelay(50 / portTICK_PERIOD_MS);
74         }
75     }
76 }
77
78 void setup() {
79     Serial.begin(115200); // Higher baud rate
80     delay(1000);
81     Serial.println("Starting FreeRTOS: Counting Semaphore");
82
83     pinMode(BUTTON_PIN, INPUT_PULLUP);
84     attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), buttonISR, FALLING);
85
86     buttonSemaphore = xSemaphoreCreateCounting(SEMAPHORE_MAX_COUNT, 0);
87     if (buttonSemaphore == NULL) {
88         Serial.println("Failed to create semaphore!");
89         while (1);
90     }
91
92     xTaskCreatePinnedToCore(
93         LEDBlinkTask,           // Task function
94         "LEDBlinkTask",        // Task name
95         3000,                   // Stack size
96         NULL,                   // Task parameters
97         2,                       // Higher priority
98         NULL,                   // Task handle
99         1                       // Core ID
100    );
101
102     xTaskCreatePinnedToCore(
103         LEDFadeTask,           // Task function
104         "LEDFadeTask",        // Task name

```



```

105     3000,           // Stack size
106     NULL,          // Task parameters
107     1,             // Lower priority
108     NULL,          // Task handle
109     1              // Core ID
110 );
111 }
112 void loop() {}

```

Queues

Queues são estruturas de dados seguras que permitem diferentes tasks receberem ou enviarem dados umas para as outras, usado para comunicação síncrona dentro de um ambiente multi-task. Também garante que não existem conflitos ou perdas de dados. A queue funciona como uma fila de espera, o primeiro a entrar é o primeiro a ser lido e removido, e os dados que estão na queue não são movidos, ou seja, copiamos a variável, isto permite-nos alterar os dados na task de origem. Para além disto, a queue elimina a necessidade de polling e contém uma espera automática.

Functions de Queue

- `xQueueCreate(size, item_size)` -> Cria uma queue, onde **size** é o número de itens/variáveis que a queue possui e **item_size** é o tamanho de bytes alocados na heap para cada item.
- `xQueueSend()` -> Adiciona dados na queue.
- `xQueueSendFromISR()` -> Adicionada dados na queue apartir de uma interrupt
- `xQueueReceive()` -> Lê dados dentro da queue, se os tiver.

Códigos de Exemplo

Básicos de Queue

```

1  /*
2   Rui Santos & Sara Santos - Random Nerd Tutorials
3   Complete project details at https://RandomNerdTutorials.com/esp32-freertos-
   queues-inter-task-arduino/
4   Permission is hereby granted, free of charge, to any person obtaining a copy of
   this software and associated documentation files.
5   The above copyright notice and this permission notice shall be included in all
   copies or substantial portions of the Software.
6  */
7  #define POT_PIN 15
8  #define LED_PIN 2
9  #define PWM_FREQ 5000
10 #define PWM_RESOLUTION 12 // 12-bit (0-4095)
11 #define QUEUE_SIZE 5
12
13 // Create an handle for the queue
14 QueueHandle_t potQueue = NULL;
15
16 void potTask(void *parameter) {
17     for (;;) {

```

```

18     uint16_t potValue = analogRead(POT_PIN); // Read 0-4095
19     xQueueSend(potQueue, &potValue, portMAX_DELAY); // Send to queue
20     Serial.printf("potTask: Sent pot value %u\n", potValue);
21     vTaskDelay(100 / portTICK_PERIOD_MS); // 100ms
22 }
23 }
24
25 void LEDBrightnessTask(void *parameter) {
26     for (;;) {
27         uint16_t potValue;
28         if (xQueueReceive(potQueue, &potValue, portMAX_DELAY)) {
29             uint16_t brightness = potValue;
30             ledcWrite(LED_PIN, brightness);
31             Serial.printf("LEDBrightnessTask: Set brightness %u\n", brightness);
32         }
33     }
34 }
35
36 void setup() {
37     Serial.begin(115200);
38
39     // Setup PWM for LED
40     ledcAttach(LED_PIN, PWM_FREQ, PWM_RESOLUTION);
41
42     // Create queue (5 items, each uint16_t)
43     potQueue = xQueueCreate(Queue_SIZE, sizeof(uint16_t));
44     if (potQueue == NULL) {
45         Serial.println("Failed to create queue!");

```

Partilha de dados entre 3 Tasks

```

1  /*
2   Rui Santos & Sara Santos - Random Nerd Tutorials
3   Complete project details at https://RandomNerdTutorials.com/esp32-freertos-
   queues-inter-task-arduino/
4   Permission is hereby granted, free of charge, to any person obtaining a copy of
   this software and associated documentation files.
5   The above copyright notice and this permission notice shall be included in all
   copies or substantial portions of the Software.
6  */
7  #define POT_PIN 15
8  #define LED_PIN 2
9  #define PWM_FREQ 5000
10 #define PWM_RESOLUTION 12 // 12-bit (0-4095)
11 #define Queue_SIZE 5
12
13 QueueHandle_t potQueue = NULL;
14 QueueHandle_t sMonitorQueue = NULL;
15
16 void SensorTask(void *parameter) {
17     for (;;) {
18         uint16_t potValue = analogRead(POT_PIN); // Read 0-4095

```

```

19     xQueueSend(potQueue, &potValue, portMAX_DELAY); // Send to queue
20     xQueueSend(sMonitorQueue, &potValue, portMAX_DELAY); // Send to queue
21     vTaskDelay(300 / portTICK_PERIOD_MS); // 300ms
22 }
23 }
24
25 void LEDBrightnessTask(void *parameter) {
26     for (;;) {
27         uint16_t potValue;
28         if (xQueueReceive(potQueue, &potValue, portMAX_DELAY)) {
29             uint16_t brightness = potValue;
30             ledcWrite(LED_PIN, brightness);
31         }
32     }
33 }
34
35 void SerialLoggerTask(void *parameter) {
36     for (;;) {
37         uint16_t potValue;
38         if (xQueueReceive(sMonitorQueue, &potValue, portMAX_DELAY)) {
39             Serial.printf("SerialLoggerTask: Pot value %u at %lu ms\n", potValue,
millis());
40         }
41     }
42 }
43
44 void setup() {
45     Serial.begin(115200);
46
47     // Setup PWM for LED
48     ledcAttach(LED_PIN, PWM_FREQ, PWM_RESOLUTION);
49
50     // Create queue (5 items, each uint16_t)
51     potQueue = xQueueCreate(Queue_SIZE, sizeof(uint16_t));
52     if (potQueue == NULL) {
53         Serial.println("Failed to create queue!");
54         while (1);
55     }
56
57     // Create queue (5 items, each uint16_t)
58     sMonitorQueue = xQueueCreate(Queue_SIZE, sizeof(uint16_t));
59     if (sMonitorQueue == NULL) {
60         Serial.println("Failed to create queue!");
61         while (1);
62     }
63
64     // Create tasks with one parameter per line
65     xTaskCreatePinnedToCore(
66         SensorTask,          // Task function
67         "SensorTask",        // Task name
68         3000,                // Stack size (bytes)
69         NULL,                // Task parameters

```

```

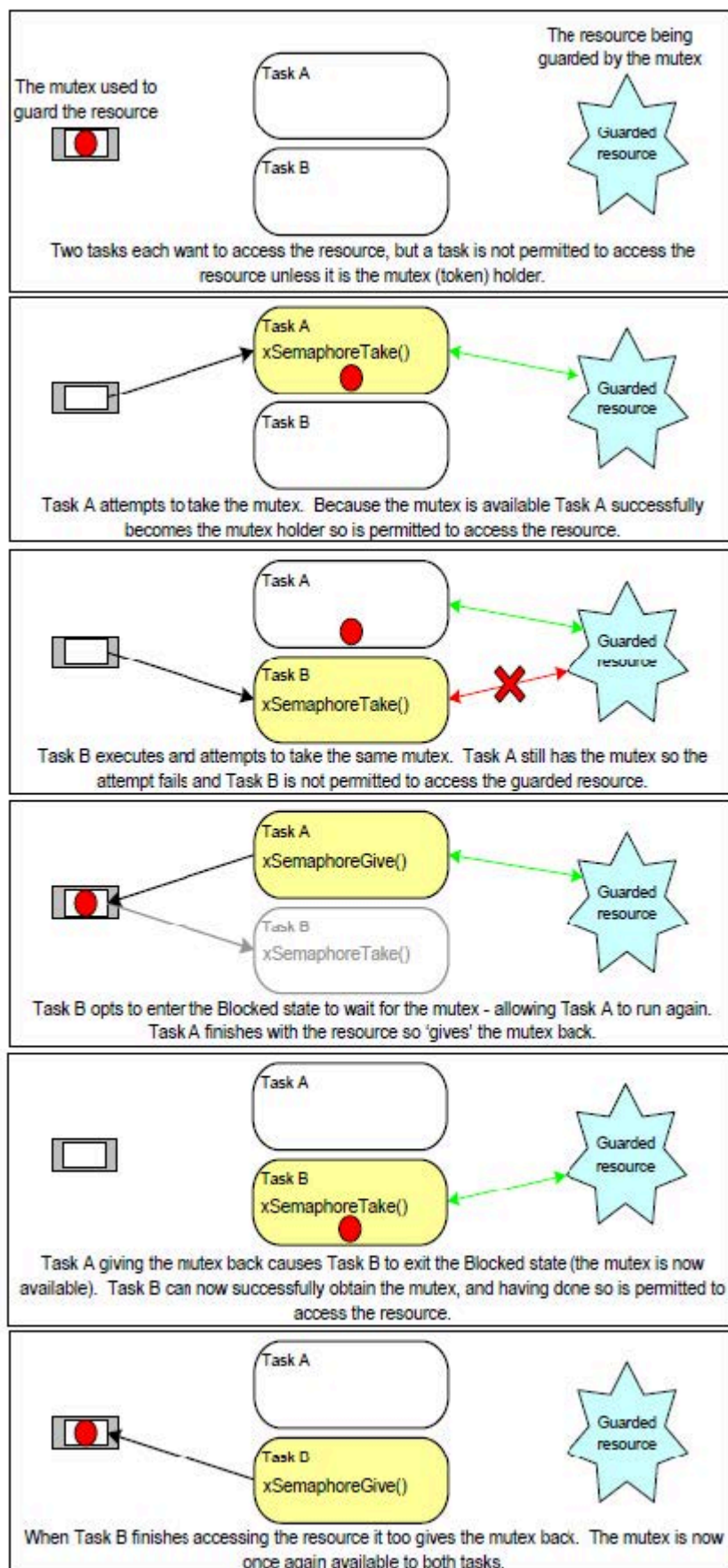
70     1, // Priority
71     NULL, // Task handle
72     1 // Core ID
73 );
74
75 xTaskCreatePinnedToCore(
76     LEDBrightnessTask, // Task function
77     "LEDBrightnessTask", // Task name
78     3000, // Stack size (bytes)
79     NULL, // Task parameters
80     1, // Priority
81     NULL, // Task handle
82     1 // Core ID
83 );
84
85 xTaskCreatePinnedToCore(
86     SerialLoggerTask, // Task function
87     "SerialLoggerTask", // Task name
88     3000, // Stack size (bytes)
89     NULL, // Task parameters
90     1, // Priority
91     NULL, // Task handle
92     1 // Core ID
93 );
94 }
95
96 void loop() {
97     // Empty
98 }

```

Mutex

Um mutex é um tipo especial de semáforo binário. Ele faz o controlo de acesso a uma fonte/variável comum a dois ou mais tasks. O termo "mutex" significa "Mutual Exclusion", funcionando como uma "chave" onde quem tem a chave tem direito a aceder à fonte/variável partilhada. Para usar um mutex, a task tem de ficar com eles, ficando com a chave e quando a task acaba ele devolve o mutex para outras task. A imagem a seguir, representa como é que duas tasks interagem quando têm uma source

em comum:



Herança de prioridade

Quando uma tarefa de alta prioridade tenta aceder a um recurso que está ocupado por uma de baixa prioridade, o FreeRTOS aplica a herança de prioridade para evitar que o sistema bloqueie. Basicamente, a tarefa que detém o Mutex herda temporariamente a prioridade da tarefa mais alta que está à espera. Isto faz com que a tarefa mais lenta termine o seu trabalho mais depressa, sem ser interrompida por outras, libertando o recurso rapidamente. Assim que o Mutex é entregue, a tarefa volta à sua prioridade normal e a tarefa de alta prioridade assume logo o controlo.

Functions de um Mutex

- `xSemaphoreCreateMutex()` -> Devolve um handle da classe `SemaphoreHandle_t` quando sucede na criação do semáforo, ou devolve NULL se não conseguir criar.
- `xSemaphoreTake(mutex, timeout)` -> Obtem um mutex e ganha acesso ao recurso partilhado
- `xSemaphoreGive(mutex)` -> Para dar o mutex, ou seja, a task que anteriormente o obteve agora "solta" o mutex para outras task acedem ao recurso partilhado.

Exemplo

Serial Output Mutex

```
1  #define MUTEX_TIMEOUT 5000 // 5s timeout
2
3  SemaphoreHandle_t serialMutex = NULL;
4
5  void Task1(void *parameter) {
6      for (;;) {
7          if (xSemaphoreTake(serialMutex, MUTEX_TIMEOUT)) {
8              Serial.println("Task1: Logging from Task 1");
9              vTaskDelay(1000 / portTICK_PERIOD_MS);
10             Serial.println("Task1: End of log from Task 1");
11             xSemaphoreGive(serialMutex);
12         }
13         vTaskDelay(5000 / portTICK_PERIOD_MS);
14     }
15 }
16
17 void Task2(void *parameter) {
18     for (;;) {
19         if (xSemaphoreTake(serialMutex, MUTEX_TIMEOUT)) {
20             Serial.println("Task2: Logging from Task 2");
21             vTaskDelay(800 / portTICK_PERIOD_MS);
22             Serial.println("Task2: End of log from Task 2");
23             xSemaphoreGive(serialMutex);
24         }
25         vTaskDelay(3000 / portTICK_PERIOD_MS);
26     }
27 }
28
29 void setup() {
30     Serial.begin(115200);
31     delay(1000);
32     Serial.println("Starting FreeRTOS");
33
34     serialMutex = xSemaphoreCreateMutex();
35     if (serialMutex == NULL) {
36         Serial.println("Failed to create mutex!");
37         while (1);
38     }
39
40     xTaskCreatePinnedToCore(
```

```

41     Task1,                // Task function
42     "Task1",              // Task name
43     3000,                 // Stack size
44     NULL,                 // Task parameters
45     1,                    // Priority
46     NULL,                 // Task handle
47     1                     // Core ID
48 );
49
50 xTaskCreatePinnedToCore(
51     Task2,                // Task function
52     "Task2",              // Task name
53     3000,                 // Stack size
54     NULL,                 // Task parameters
55     2,                    // Higher priority
56     NULL,                 // Task handle
57     1                     // Core ID
58 );
59 }
60
61 void loop() {
62
63 }

```

Referência

<https://randomnerdtutorials.com/esp32-freertos-arduino-tasks/>

<https://randomnerdtutorials.com/esp32-freertos-semaphores-arduino/>

<https://randomnerdtutorials.com/esp32-freertos-queues-inter-task-arduino/#more-173794>

<https://randomnerdtutorials.com/esp32-freertos-mutex-arduino/>